Práctica N°1: Inicializacion de OLED y perifericos en PIC32 MAX32

Índice

1. Objetivos **3**
- 1.1. Objetivo general . . . 3
- 1.2. Objetivos específicos . . . 3

2. Instalación del entorno de desarrollo **3**
- 2.1. Instalación de Visual Studio Code . . . 4
- 2.2. Instalación de la extensión PlatformIO IDE . . . 4
- 2.3. Creación de un nuevo proyecto para ESP32 . . . 5
- 2.4. Instalación de la librería `FastLED` . . . 5
  - 2.4.1. Método 1: desde el gestor de librerías de PlatformIO . . . 5
  - 2.4.2. Método 2: agregando la dependencia en `platformio.ini` 6
- 2.5. Explicación del código por bloques . . . 7
  - 2.5.1. Código completo . . . 7
  - 2.5.2. Inclusiones y definiciones principales . . . 14
  - 2.5.3. Declaración del buffer y variables de control . . . 15
  - 2.5.4. Mapeo XY (serpentina) . . . 15
  - 2.5.5. Fuente 5x7 y helper de índice . . . 16
  - 2.5.6. Efecto: rainbowChase . . . 16
  - 2.5.7. Efecto: plasma . . . 17
  - 2.5.8. Efecto: fireworks . . . 17
  - 2.5.9. Efecto: checkerboard . . . 18
  - 2.5.10. Utilidades de dibujo y texto . . . 19
  - 2.5.11. Scroll de texto . . . 20
  - 2.5.12. Mensaje de bienvenida . . . 21
  - 2.5.13. ISR del botón . . . 22
  - 2.5.14. setup() . . . 22
  - 2.5.15. loop() . . . 23

3. Carga del programa en la placa y verificación **23**
3.1. Comprobaciones previas 24
3.2. Compilar el proyecto (Build) 24
3.3. Seleccionar puerto serie y subir (Upload) 25

4. Resultados **26**

1. Objetivos

1.1. Objetivo general

- Diseñar, implementar y documentar una guía práctica que permita a un estudiante instalar y usar PlatformIO en Visual Studio Code para programar una matriz LED 16×16 basada en WS2812 con un ESP32, de modo que pueda desplegar efectos visuales (rainbow, plasma, fuegos artificiales, tablero ajedrez) y texto desplazable con funcionamiento seguro y reproducible.

1.2. Objetivos específicos

1. **Instalación del entorno:** Mostrar paso a paso la instalación de Visual Studio Code y la extensión PlatformIO, crear un proyecto nuevo para ESP32 y verificar la conexión entre la placa y PlatformIO (compilación y carga de un sketch de prueba).

2. **Gestión de dependencias:** Incluir las instrucciones para agregar la librería `FastLED` en `platformio.ini` y verificar su correcto funcionamiento mediante un ejemplo mínimo.

3. **Portado y explicación del código:** Incluir el código proporcionado (mapeo XY serpentina, fuente 5x7, efectos y scroll) con explicaciones linea-por-línea de las partes críticas: inicialización de FastLED, `XY()`, ISR del botón, y gestión de modos.

4. **Pruebas en hardware:** Definir procedimientos de prueba (lista de verificación) para comprobar el mapeo, la orientación de la matriz, el brillo seguro y la respuesta del botón (debounce). Incluir criterios de aceptación (por ejemplo: todas las filas/columnas deben encenderse correctamente con el patrón de test; el cambio de modo debe ser estable y reproducible).

2. Instalación del entorno de desarrollo

En esta sección se detallan los pasos necesarios para preparar el entorno de desarrollo basado en **Visual Studio Code (VSCode)** con la extensión **PlatformIO**, que permite programar el microcontrolador **ESP32** de forma profesional, integrando herramientas de compilación, carga y gestión de librerías.

2.1. Instalación de Visual Studio Code

1. Acceder al sitio oficial de descarga: https://code.visualstudio.com/.
2. Descargar la versión correspondiente al sistema operativo Windows.
3. Ejecutar el instalador y seguir los pasos del asistente, dejando activadas las opciones recomendadas.

Una vez completada la instalación, abrir VSCode para verificar que la interfaz inicial se muestra correctamente.

2.2. Instalación de la extensión PlatformIO IDE

1. En la barra lateral izquierda de VSCode, hacer clic en el icono de **Extensiones**.
2. En la barra de búsqueda escribir: `PlatformIO IDE`.
3. Seleccionar la extensión oficial de **PlatformIO Labs LLC**.
4. Pulsar en el botón **Install**.

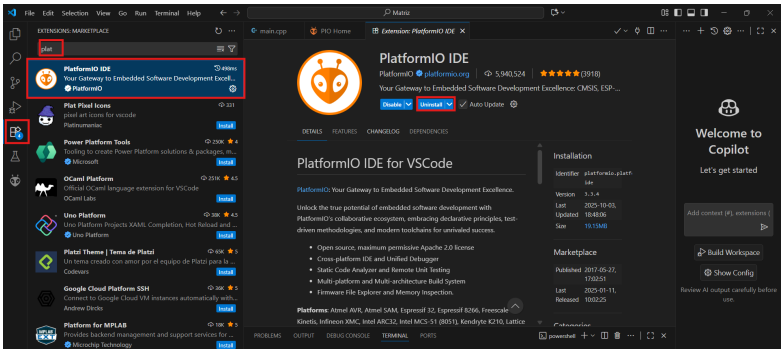


Figura 1: Instalación de la extensión PlatformIO IDE desde el marketplace de VSCode.

Tras la instalación, PlatformIO añadirá un icono con el logotipo del "hormiga" (**ant**) en la barra lateral izquierda. Al hacer clic sobre él se accede al panel de control principal.

2.3. Creación de un nuevo proyecto para ESP32

1. En el menú lateral de PlatformIO, seleccionar **New Project**.
2. Asignar un nombre al proyecto (por ejemplo: `Matriz_LED_ESP32`).
3. En **Board**, seleccionar la placa utilizada, por ejemplo `ESP32 Dev Module`.
4. En **Framework**, elegir `Arduino`.
5. Pulsar **Finish** y esperar a que PlatformIO configure automáticamente la estructura del proyecto.

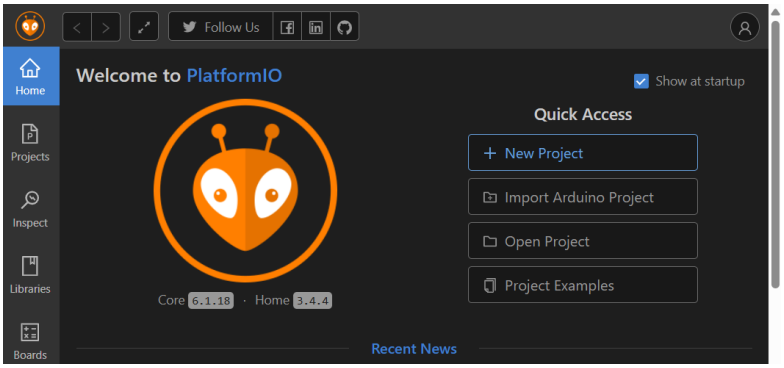


Figura 2: Creación de un nuevo proyecto en PlatformIO para ESP32 con el framework Arduino.

2.4. Instalación de la librería `FastLED`

La librería `FastLED` es necesaria para controlar las matrices de LED direccionables como las WS2812. Existen dos métodos para instalarla: usando la interfaz de PlatformIO o editando el archivo de configuración.

2.4.1. Método 1: desde el gestor de librerías de PlatformIO

1. Hacer clic en el icono de PlatformIO (barra lateral izquierda).
2. Seleccionar la pestaña **Libraries**.
3. Escribir en el buscador: `FastLED`.
4. Seleccionar la librería oficial de **Daniel Garcia**.
5. Pulsar **Install**.

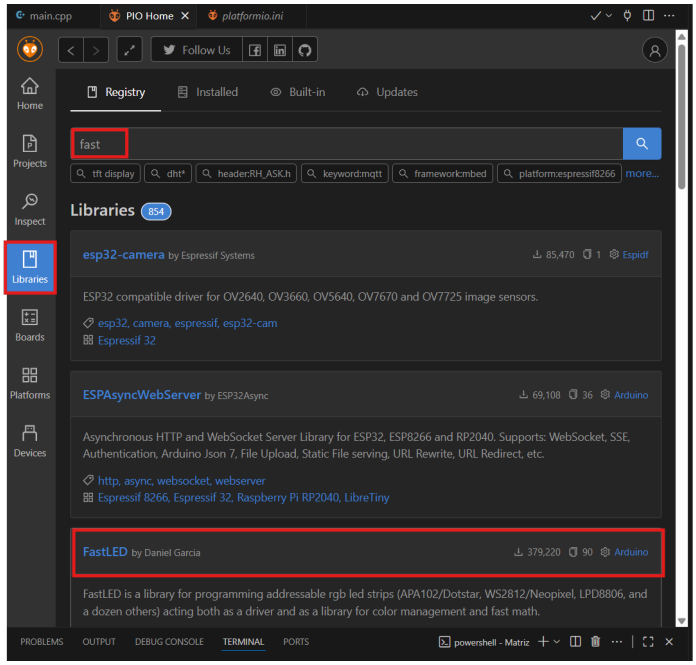


Figura 3: Instalación de la librería `FastLED` desde el gestor de librerías de PlatformIO.

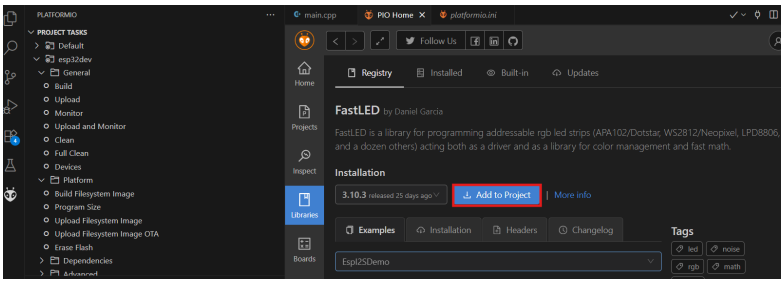


Figura 4: Instalación de la librería `FastLED` desde el gestor de librerías de PlatformIO.

2.4.2. Método 2: agregando la dependencia en `platformio.ini`

Otra forma de instalar la librería es editando manualmente el archivo de configuración `platformio.ini` del proyecto, añadiendo la línea:

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
lib_deps = FastLED
```

Listing 1: Configuracion de librerias en platformio.ini

Al guardar el archivo, PlatformIO descargará automáticamente la librería la próxima vez que se compile el proyecto.

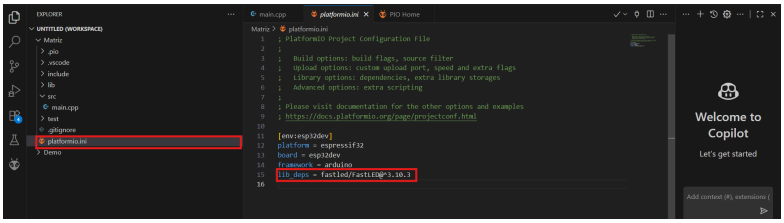


Figura 5: Ejemplo de archivo `platformio.ini` con la dependencia de `FastLED` añadida.

2.5. Explicación del código por bloques

2.5.1. Código completo

```
#include <Arduino.h>
// Matriz 16x16 WS2812 con ESP32 (FastLED)
// Data: GPIO4 por defecto. Cambia segn tu conexin.

#include <FastLED.h>

#define MATRIX_WIDTH 16
#define MATRIX_HEIGHT 16
#define NUM_LEDS (MATRIX_WIDTH * MATRIX_HEIGHT)
#define DATA_PIN 4      // pin de datos al primer LED
#define BUTTON_PIN 2    // cambiar si usas diferente
#define BRIGHTNESS 120  // reduccin para no sobrecargar la fuente

CRGB leds[NUM_LEDS];

// --- Configuracin de modos / botones
volatile uint8_t mode = 0;
const uint8_t MODES = 5;
```

```
19 unsigned long lastButtonMillis = 0;
20 const unsigned long debounce = 200;
21
22 // --- Variables de control de modos (para reiniciar estado al
       cambiar de modo)
23 uint8_t prevMode = 255;
24 bool showHolaDone = false;
25
26 // --- Mapeo XY para matriz serpentina (zig-zag)
27 inline uint16_t XY(uint8_t x, uint8_t y){
28   if(x >= MATRIX_WIDTH || y >= MATRIX_HEIGHT) return 0;
29   if(y % 2 == 0) {
30     return (y * MATRIX_WIDTH) + (MATRIX_WIDTH - 1 - x);
31   } else {
32     return (y * MATRIX_WIDTH) + x;
33   }
34 }
35
36 // ----------------- Fuente 5x7 (A-Z + espacio) -----------------
37 const uint8_t font5x7[][5] = {
38   {0x7C,0x12,0x11,0x12,0x7C}, // A
39   {0x7F,0x49,0x49,0x49,0x36}, // B
40   {0x3E,0x41,0x41,0x41,0x22}, // C
41   {0x7F,0x41,0x41,0x22,0x1C}, // D
42   {0x7F,0x49,0x49,0x49,0x41}, // E
43   {0x7F,0x09,0x09,0x09,0x01}, // F
44   {0x3E,0x41,0x49,0x49,0x7A}, // G
45   {0x7F,0x08,0x08,0x08,0x7F}, // H
46   {0x00,0x41,0x7F,0x41,0x00}, // I
47   {0x20,0x40,0x41,0x3F,0x01}, // J
48   {0x7F,0x08,0x14,0x22,0x41}, // K
49   {0x7F,0x40,0x40,0x40,0x40}, // L
50   {0x7F,0x02,0x0C,0x02,0x7F}, // M
51   {0x7F,0x04,0x08,0x10,0x7F}, // N
52   {0x3E,0x41,0x41,0x41,0x3E}, // O
53   {0x7F,0x09,0x09,0x09,0x06}, // P
54   {0x3E,0x41,0x51,0x21,0x5E}, // Q
55   {0x7F,0x09,0x19,0x29,0x46}, // R
56   {0x46,0x49,0x49,0x49,0x31}, // S
57   {0x01,0x01,0x7F,0x01,0x01}, // T
58   {0x3F,0x40,0x40,0x40,0x3F}, // U
59   {0x1F,0x20,0x40,0x20,0x1F}, // V
60   {0x3F,0x40,0x38,0x40,0x3F}, // W
```

```
  {0x63,0x14,0x08,0x14,0x63}, // X
  {0x07,0x08,0x70,0x08,0x07}, // Y
  {0x61,0x51,0x49,0x45,0x43}, // Z
  {0x00,0x00,0x00,0x00,0x00} // espacio (index 26)
};

inline int fontIndex(char c){
  if(c >= 'A' && c <= 'Z') return c - 'A';
  if(c == ' ') return 26;
  return 26;
}

// --- Efectos originales (sin tocar mucho)
void rainbowChase(uint8_t wait){
  static uint8_t startHue = 0;
  for(uint8_t y=0;y<MATRIX_HEIGHT;y++){
    for(uint8_t x=0;x<MATRIX_WIDTH;x++){
      uint16_t i = XY(x,y);
      uint8_t hue = startHue + (x*8) + (y*4);
      leds[i] = CHSV(hue, 200, 255);
    }
  }
  startHue += 1;
  FastLED.show();
  delay(wait);
}

void plasma(uint8_t t){
  static uint16_t count = 0;
  for(uint8_t y=0;y<MATRIX_HEIGHT;y++){
    for(uint8_t x=0;x<MATRIX_WIDTH;x++){
      float v = sinf((x+count)/4.0f) + sinf((y-count)/3.0f) +
          sinf((x+y+count)/5.0f);
      uint8_t hue = (uint8_t)map((int)((v+3.0f)*40.0f), 0, 240, 0,
          255);
      leds[XY(x,y)] = CHSV(hue, 220, 200);
    }
  }
  count += t;
  FastLED.show();
  delay(20);
}

```

```
102 void fireworks(){
103   static CRGB fade[NUM_LEDS];
104   // decay: 250 es rpido, 200 es ms lento (ms  fcil  de ver)
105   const uint8_t decay = 200;
106   for(int i=0;i<NUM_LEDS;i++){
107     fade[i].nscale8(decay); // reduce brillo de cada frame
108   }
109 
110   const int spawnRate = 20; // prueba con 20 (ms frecuente). Pon 5
        para muchas.
111   if(random(0, spawnRate) == 0){
112     int cx = random(2, MATRIX_WIDTH - 2);
113     int cy = random(2, MATRIX_HEIGHT - 2);
114     CRGB color = CHSV(random8(), 255, 255);
115     for(int dx = -2; dx <= 2; dx++){
116       for(int dy = -2; dy <= 2; dy++){
117         int x = cx + dx;
118         int y = cy + dy;
119         if(x >= 0 && x < MATRIX_WIDTH && y >= 0 && y <
              MATRIX_HEIGHT){
120           float dist = sqrtf((float)(dx*dx + dy*dy));
121           int bright = 255 - int(dist * 90.0f);
122           if(bright > 0){
123             CRGB scaled = color;
124             scaled.nscale8((uint8_t)bright);
125             fade[XY(x,y)] += scaled;
126           }
127         }
128       }
129     }
130   }
131 
132   for(int i=0;i<NUM_LEDS;i++) leds[i] = fade[i];
133   FastLED.show();
134   delay(35);
135 }
136 
137 void checkerboard(uint8_t t){
138   for(uint8_t y=0;y<MATRIX_HEIGHT;y++){
139     for(uint8_t x=0;x<MATRIX_WIDTH;x++){
140       bool on = ((x/2 + y/2 + (t/10)) % 2) == 0;
141       uint16_t idx = XY(x,y);
142       if(on){
```

```
143         leds[idx] = CHSV( (uint8_t)(160 + (x*3)), 200, 255 );
144       } else {
145         leds[idx] = CRGB::Black;
146       }
147     }
148   }
149   FastLED.show();
150   delay(40);
151 }
152 
153 // ----------------- Funciones de texto y scroll -----------------
154 void clearMatrix(){
155   for(int i=0;i<NUM_LEDS;i++) leds[i] = CRGB::Black;
156 }
157 
158 inline void drawPixelSafe(int x, int y, CRGB color){
159   if(x < 0 || x >= MATRIX_WIDTH || y < 0 || y >= MATRIX_HEIGHT)
        return;
160   leds[XY(x,y)] = color;
161 }
162 
163 void drawTextAt(int x0, const char* text, CRGB color){
164   int x = x0;
165   for(size_t i=0; text[i] != '\0'; ++i){
166     char c = text[i];
167     if(c >= 'a' && c <= 'z') c = c - 'a' + 'A';
168     int idx = fontIndex(c);
169     const uint8_t *glyph = font5x7[idx];
170     for(int col=0; col<5; ++col){
171       uint8_t columnBits = glyph[col];
172       for(int row=0; row<7; ++row){
173         if(columnBits & (1<<row)){
174           int y = row + (MATRIX_HEIGHT - 7)/2;
175           drawPixelSafe(x + col, y, color);
176         }
177       }
178     }
179     x += 6; // 5 columnas + 1 columna de espacio
180   }
181 }
182 
183 void scrollMessage(const char* text, uint8_t speed, CHSV colorHSV){
184   int len = strlen(text);
```

```
185   int totalCols = len * 6;
186   uint8_t *cols = (uint8_t*)malloc(totalCols);
187   if(!cols) return;
188   int colIdx = 0;
189   for(int i=0;i<len;i++){
190     char c = text[i];
191     if(c >= 'a' && c <= 'z') c = c - 'a' + 'A';
192     int fi = fontIndex(c);
193     const uint8_t *g = font5x7[fi];
194     for(int ccol=0; ccol<5; ++ccol){
195       cols[colIdx++] = g[ccol];
196     }
197     cols[colIdx++] = 0x00;
198   }
199
200   int scrollRange = totalCols + MATRIX_WIDTH;
201   for(int off = 0; off < scrollRange; ++off){
202     clearMatrix();
203     for(int x=0; x < MATRIX_WIDTH; ++x){
204       int sourceCol = off + x - MATRIX_WIDTH;
205       if(sourceCol >= 0 && sourceCol < totalCols){
206         uint8_t columnBits = cols[sourceCol];
207         for(int row=0; row<7; ++row){
208           if(columnBits & (1<<row)){
209             int y = row + (MATRIX_HEIGHT - 7)/2;
210             drawPixelSafe(x, y, colorHSV);
211           }
212         }
213       }
214     }
215     FastLED.show();
216     delay(speed);
217   }
218
219   free(cols);
220 }
221
222 void showHola(unsigned long duration = 2000){
223   const char *msg = "HOLA";
224   int len = strlen(msg);
225   int width = len * 6 - 1;
226   int x0 = (MATRIX_WIDTH - width) / 2;
227   if(x0 < 0) x0 = 0;
```

```
228   clearMatrix();
229   drawTextAt(x0, msg, CHSV(160, 200, 255));
230   FastLED.show();
231   delay(duration);
232 }
233
234 // ----------------- Interrupcin de botn (secuencial de modos)
      -----------------
235 void IRAM_ATTR handleButton(){
236   unsigned long now = millis();
237   if(now - lastButtonMillis < debounce) return;
238   lastButtonMillis = now;
239   mode = (mode + 1) % MODES;
240 }
241
242 // ----------------- Setup / Loop -----------------
243 void setup(){
244   Serial.begin(115200);
245   delay(100);
246   FastLED.addLeds<WS2812, DATA_PIN, GRB>(leds,
        NUM_LEDS).setCorrection(TypicalLEDStrip);
247   FastLED.setBrightness(BRIGHTNESS);
248   pinMode(BUTTON_PIN, INPUT_PULLUP);
249   attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), handleButton,
        FALLING);
250   randomSeed(analogRead(36));
251   clearMatrix();
252   FastLED.show();
253 }
254
255 void loop(){
256   if(prevMode != mode){
257     prevMode = mode;
258     showHolaDone = false;
259   }
260
261   switch(mode){
262     case 0:
263       rainbowChase(20);
264       break;
265     case 1:
266       plasma(1);
267       break;
```

```
    case 2:
      fireworks();
      break;
    case 3:
      checkerboard(millis()/50);
      break;
    case 4:
      scrollMessage("UPS CUENCA", 60, CHSV(0, 0, 255));
      break;
    default:
      mode = 0;
  }
}
```

Listing 2: Código completo para la matriz LED 16x16 con ESP32 y FastLED

2.5.2. Inclusiones y definiciones principales

```
#include <Arduino.h>
#include <FastLED.h>

#define MATRIX_WIDTH 16
#define MATRIX_HEIGHT 16
#define NUM_LEDS (MATRIX_WIDTH * MATRIX_HEIGHT)
#define DATA_PIN 4
#define BUTTON_PIN 2
#define BRIGHTNESS 120
```

Listing 3: Inclusiones y macros principales

Este bloque carga las librerías esenciales y define parámetros constantes: tamaño de la matriz, número total de LEDs, pines de datos y del botón, y el brillo máximo. Las macros facilitan cambiar dimensiones o pines en un solo lugar sin buscar en todo el código. El valor de brillo se reduce para limitar la corriente consumida por la matriz.

2.5.3. Declaración del buffer y variables de control

```
CRGB leds[NUM_LEDS];

volatile uint8_t mode = 0;
const uint8_t MODES = 5;
unsigned long lastButtonMillis = 0;
const unsigned long debounce = 200;

uint8_t prevMode = 255;
bool showHolaDone = false;
```

Listing 4: Buffer de LEDs y variables de control

Aquí se reserva el arreglo que representa cada LED en memoria (formato CRGB). También se declaran variables que controlan el modo actual, el número total de modos, y el mecanismo de debounce para evitar lecturas múltiples del botón por rebote. ‘prevMode‘ y ‘showHolaDone‘ sirven para detectar cambios de modo y manejar acciones que sólo deben ejecutarse al entrar en un modo.

2.5.4. Mapeo XY (serpentina)

```
inline uint16_t XY(uint8_t x, uint8_t y){
  if(x >= MATRIX_WIDTH || y >= MATRIX_HEIGHT) return 0;
  if(y % 2 == 0) {
    return (y * MATRIX_WIDTH) + (MATRIX_WIDTH - 1 - x);
  } else {
    return (y * MATRIX_WIDTH) + x;
  }
}
```

Listing 5: Función de mapeo XY para matriz serpentina

Convierte coordenadas (x,y) en el índice lineal usado por el buffer ‘leds[]‘, corrigiendo la disposición física serpentina (zig-zag). Si la fila es par, se invierte el orden horizontal para coincidir con el cableado típico de matrices WS2812; si la fila es impar, se usa el orden natural. La validación inicial evita accesos fuera de rango.

2.5.5. Fuente 5x7 y helper de índice

```
const uint8_t font5x7[][5] = {
  {0x7C,0x12,0x11,0x12,0x7C}, // A
  {0x7F,0x49,0x49,0x49,0x36}, // B
  ...
  {0x00,0x00,0x00,0x00,0x00} // espacio
};

inline int fontIndex(char c){
  if(c >= 'A' && c <= 'Z') return c - 'A';
  if(c == ' ') return 26;
  return 26;
}
```

Listing 6: Fuente 5x7 y función auxiliar

Contiene las columnas de bits por carácter para una fuente de 5×7. Cada byte representa una columna vertical de 7 píxeles; cinco bytes forman un glifo. La función auxiliar mapea un carácter ASCII (A–Z o espacio) al índice correcto del arreglo. Si el carácter no se reconoce, se devuelve el índice del espacio.

2.5.6. Efecto: rainbowChase

```
void rainbowChase(uint8_t wait){
  static uint8_t startHue = 0;
  for(uint8_t y=0;y<MATRIX_HEIGHT;y++){
    for(uint8_t x=0;x<MATRIX_WIDTH;x++){
      uint16_t i = XY(x,y);
      uint8_t hue = startHue + (x*8) + (y*4);
      leds[i] = CHSV(hue, 200, 255);
    }
  }
  startHue += 1;
  FastLED.show();
  delay(wait);
}
```

Listing 7: Efecto rainbowChase

Genera un gradiente de color en modo HSV donde el tono depende de la posición x,y y de un desplazamiento ‘startHue‘ que cambia en el tiempo para animar el arcoíris. ‘CHSV‘ simplifica la manipulación de color y

‘FastLED.show()‘ actualiza físicamente la matriz. El parámetro ‘wait‘ regula la velocidad del efecto.

2.5.7. Efecto: plasma

```
void plasma(uint8_t t){
  static uint16_t count = 0;
  for(uint8_t y=0;y<MATRIX_HEIGHT;y++){
    for(uint8_t x=0;x<MATRIX_WIDTH;x++){
      float v = sinf((x+count)/4.0f) + sinf((y-count)/3.0f) +
          sinf((x+y+count)/5.0f);
      uint8_t hue = (uint8_t)map((int)((v+3.0f)*40.0f), 0, 240, 0,
          255);
      leds[XY(x,y)] = CHSV(hue, 220, 200);
    }
  }
  count += t;
  FastLED.show();
  delay(20);
}
```

Listing 8: Efecto plasma

Crea un efecto ondulante combinando varias funciones seno para obtener variaciones suaves en el valor ‘v‘. Ese valor se mapea a un tono HSV que produce las transiciones de color tipo plasma. La variable ‘count‘ avanza con el tiempo para desplazar la animación; el parámetro ‘t‘ controla la velocidad de avance.

2.5.8. Efecto: fireworks

```
void fireworks(){
  static CRGB fade[NUM_LEDS];
  const uint8_t decay = 200;
  for(int i=0;i<NUM_LEDS;i++){
    fade[i].nscale8(decay);
  }

  const int spawnRate = 20;
  if(random(0, spawnRate) == 0){
    int cx = random(2, MATRIX_WIDTH - 2);
    int cy = random(2, MATRIX_HEIGHT - 2);
    CRGB color = CHSV(random8(), 255, 255);
```

```
    for(int dx = -2; dx <= 2; dx++){
      for(int dy = -2; dy <= 2; dy++){
        int x = cx + dx;
        int y = cy + dy;
        if(x >= 0 && x < MATRIX_WIDTH && y >= 0 && y <
            MATRIX_HEIGHT){
          float dist = sqrtf((float)(dx*dx + dy*dy));
          int bright = 255 - int(dist * 90.0f);
          if(bright > 0){
            CRGB scaled = color;
            scaled.nscale8((uint8_t)bright);
            fade[XY(x,y)] += scaled;
          }
        }
      }
    }
  }

  for(int i=0;i<NUM_LEDS;i++) leds[i] = fade[i];
  FastLED.show();
  delay(35);
}
```

Listing 9: Efecto fireworks

Mantiene un buffer ‘fade[]‘ que decae cada frame (escala hacia abajo el brillo). Ocasionalmente se genera una explosión en una posición aleatoria, pintando píxeles en un radio y sumando su contribución al buffer. El decaimiento hace que las explosiones se atenúen con el tiempo, creando el efecto de fuegos artificiales.

2.5.9. Efecto: checkerboard

```
void checkerboard(uint8_t t){
  for(uint8_t y=0;y<MATRIX_HEIGHT;y++){
    for(uint8_t x=0;x<MATRIX_WIDTH;x++){
      bool on = ((x/2 + y/2 + (t/10)) % 2) == 0;
      uint16_t idx = XY(x,y);
      if(on){
        leds[idx] = CHSV( (uint8_t)(160 + (x*3)), 200, 255 );
      } else {
        leds[idx] = CRGB::Black;
      }
    }
```

```
  }
  FastLED.show();
  delay(40);
}
```

Listing 10: Efecto checkerboard

Dibuja un tablero de cuadros alternados donde el patrón se desplaza en el tiempo (la expresión ‘(t/10)‘ añade animación). La condición ‘on‘ decide si una celda está encendida; el color varía ligeramente por columna para dar dinamismo.

2.5.10. Utilidades de dibujo y texto

```
void clearMatrix(){
  for(int i=0;i<NUM_LEDS;i++) leds[i] = CRGB::Black;
}
inline void drawPixelSafe(int x, int y, CRGB color){
  if(x < 0 || x >= MATRIX_WIDTH || y < 0 || y >= MATRIX_HEIGHT)
      return;
  leds[XY(x,y)] = color;
}
void drawTextAt(int x0, const char* text, CRGB color){
  int x = x0;
  for(size_t i=0; text[i] != '\0'; ++i){
    char c = text[i];
    if(c >= 'a' && c <= 'z') c = c - 'a' + 'A';
    int idx = fontIndex(c);
    const uint8_t *glyph = font5x7[idx];
    for(int col=0; col<5; ++col){
      uint8_t columnBits = glyph[col];
      for(int row=0; row<7; ++row){
        if(columnBits & (1<<row)){
          int y = row + (MATRIX_HEIGHT - 7)/2;
          drawPixelSafe(x + col, y, color);
        }
      }
    }
    x += 6;
  }
}
```

Listing 11: Funciones clearMatrix, drawPixelSafe, drawTextAt

‘clearMatrix()‘ pone todos los LEDs a negro. ‘drawPixelSafe()‘ escribe un pixel solo si está dentro de los límites, evitando accesos inválidos. ‘drawTextAt()‘ recorre cada carácter, extrae su glifo en columnas y pinta las columnas en la posición horizontal indicada; hace conversión a mayúsculas para simplificar el mapeo con la tabla de fuente.

2.5.11. Scroll de texto

```
void scrollMessage(const char* text, uint8_t speed, CHSV colorHSV){
  int len = strlen(text);
  int totalCols = len * 6;
  uint8_t *cols = (uint8_t*)malloc(totalCols);
  if(!cols) return;
  int colIdx = 0;
  for(int i=0;i<len;i++){
    char c = text[i];
    if(c >= 'a' && c <= 'z') c = c - 'a' + 'A';
    int fi = fontIndex(c);
    const uint8_t *g = font5x7[fi];
    for(int ccol=0; ccol<5; ++ccol){
      cols[colIdx++] = g[ccol];
    }
    cols[colIdx++] = 0x00;
  }

  int scrollRange = totalCols + MATRIX_WIDTH;
  for(int off = 0; off < scrollRange; ++off){
    clearMatrix();
    for(int x=0; x < MATRIX_WIDTH; ++x){
      int sourceCol = off + x - MATRIX_WIDTH;
      if(sourceCol >= 0 && sourceCol < totalCols){
        uint8_t columnBits = cols[sourceCol];
        for(int row=0; row<7; ++row){
          if(columnBits & (1<<row)){
            int y = row + (MATRIX_HEIGHT - 7)/2;
            drawPixelSafe(x, y, colorHSV);
          }
        }
      }
    }
    FastLED.show();
    delay(speed);
  }
```

```
36
37   free(cols);
38 }
```

Listing 12: Función scrollMessage

Construye un buffer de columnas (5 columnas por carácter + 1 columna vacía) y recorre ese buffer desplazándolo sobre la matriz. Para cada desplazamiento se limpia la matriz, se calculan las columnas visibles y se pintan los píxeles según los bits de cada columna. ‘speed‘ controla el retardo entre pasos y, por tanto, la velocidad del scroll. Se usa ‘malloc‘ y ‘free‘ para el buffer; en sistemas con memoria limitada, considerar usar un buffer estático o streaming para evitar fragmentación.

2.5.12. Mensaje de bienvenida

```
1  void showHola(unsigned long duration = 2000){
2    const char *msg = "HOLA";
3    int len = strlen(msg);
4    int width = len * 6 - 1;
5    int x0 = (MATRIX_WIDTH - width) / 2;
6    if(x0 < 0) x0 = 0;
7    clearMatrix();
8    drawTextAt(x0, msg, CHSV(160, 200, 255));
9    FastLED.show();
10   delay(duration);
11 }
```

Listing 13: Función showHola

Centra el texto "HOLA.[en] la matriz calculando el ancho en columnas y usando ‘drawTextAt()‘ para dibujarlo. Se muestra durante el tiempo indicado por ‘duration‘, útil como introducción al iniciar o al cambiar de modo.

2.5.13. ISR del botón

```
void IRAM_ATTR handleButton(){
  unsigned long now = millis();
  if(now - lastButtonMillis < debounce) return;
  lastButtonMillis = now;
  mode = (mode + 1) % MODES;
}
```

Listing 14: Rutina de interrupción handleButton

Rutina que se ejecuta en respuesta a la señal del botón. Implementa debounce temporal (ignorando pulsos más rápidos que debounce) y avanza la variable mode de forma cíclica. La etiqueta IRAM_ATTR sugiere que la función se coloque en la IRAM del ESP32 para una respuesta más fiable en interrupciones.

2.5.14. setup()

```
void setup(){
  Serial.begin(115200);
  delay(100);
  FastLED.addLeds<WS2812, DATA_PIN, GRB>(leds,
      NUM_LEDS).setCorrection(TypicalLEDStrip);
  FastLED.setBrightness(BRIGHTNESS);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), handleButton,
      FALLING);
  randomSeed(analogRead(36));
  clearMatrix();
  FastLED.show();
}
```

Listing 15: Función setup

Inicializa la comunicación serial, configura la librería FastLED con el tipo de LEDs y el buffer, ajusta brillo, configura el pin del botón con pull-up interno y enlaza la interrupción al pin. También inicializa la semilla aleatoria y limpia la matriz antes de comenzar.

2.5.15. loop()

```
void loop(){
  if(prevMode != mode){
    prevMode = mode;
    showHolaDone = false;
  }

  switch(mode){
    case 0:
      rainbowChase(20);
      break;
    case 1:
      plasma(1);
      break;
    case 2:
      fireworks();
      break;
    case 3:
      checkerboard(millis()/50);
      break;
    case 4:
      scrollMessage("UPS CUENCA", 60, CHSV(0, 0, 255));
      break;
    default:
      mode = 0;
  }
}
```

Listing 16: Bucle principal loop

Bucle principal que detecta cambios de modo para poder reiniciar estados y luego ejecuta el efecto correspondiente según ‘mode‘. Cada caso llama a la función que dibuja y actualiza la matriz; el flujo permanece en esa función hasta que ésta retorna y el bucle se vuelve a ejecutar (el botón puede cambiar ‘mode‘ desde la ISR en cualquier momento).

3. Carga del programa en la placa y verificación

Esta sección explica paso a paso cómo compilar el proyecto en PlatformIO (VSCode) y cargarlo en la placa ESP32, además de cómo verificar que el

programa se ejecuta correctamente mediante pruebas en la matriz LED y la consola serial.

3.1. Comprobaciones previas

Antes de compilar y cargar:

- Asegurar que la placa ESP32 está conectada al ordenador por USB y que la masa (GND) comparte referencia con la fuente de la matriz LED si ésta tiene alimentación externa.
- Revisar el archivo `platformio.ini` y confirmar que la seccion `env` usa la placa correcta (por ejemplo, `board = esp32dev`) y el framework `arduino`.
- Instalar drivers USB si el sistema operativo los solicita (por ejemplo CP210x o CH340, según la placa).
- Verificar que `FastLED` está añadida en `lib_deps` o instalada desde el gestor de librerías de PlatformIO.

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
lib_deps =
  FastLED
build_flags =
  ; opciones adicionales si se requieren
```

Listing 17: Ejemplo mínimo de `platformio.ini`

3.2. Compilar el proyecto (Build)

1. Abrir el proyecto en VSCode y seleccionar el icono de PlatformIO en la barra lateral.
2. Pulsar el botón **Build** (icono de check / "Build") o usar el atajo en la barra inferior de PlatformIO.
3. Observar la consola de salida: PlatformIO descargará dependencias (si es la primera vez) y compilará el proyecto.

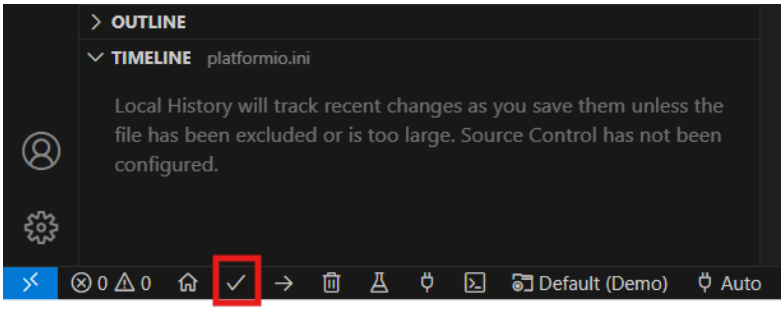


Figura 6: Compilación del proyecto en PlatformIO — salida en la pestaña *Terminal / Output*.

Si la compilación falla, revisar errores en la consola. Los mensajes habituales incluyen librerías faltantes, errores de sintaxis o definiciones de pines inconsistentes.

3.3. Seleccionar puerto serie y subir (Upload)

1. Con la placa conectada, identificar el puerto COM (Windows) o `/dev/ttyUSB*` (Linux/macOS). En VSCode → PlatformIO → **Devices** suele aparecer el puerto detectado.

2. Si el puerto no aparece, instalar el driver USB correcto o probar con otro cable USB (muchos cables son solo de carga).

3. Pulsar el botón **Upload** (icono de flecha) en PlatformIO o usar el comando `PlatformIO: Upload` desde la paleta de comandos.

4. Observar la salida: PlatformIO compilará (si es necesario) y ejecutará el proceso de subida (esptool). Esperar a que indique `SUCCESS` o `DONE`.

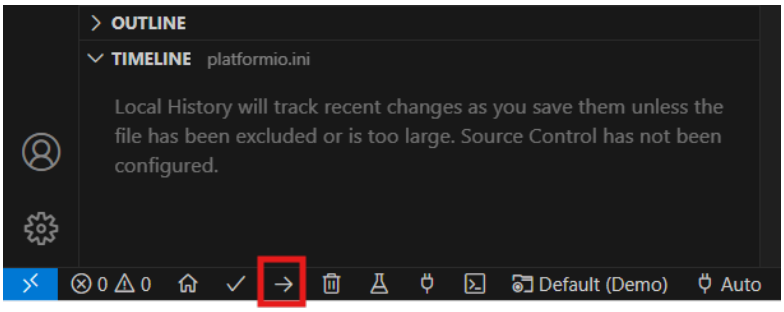


Figura 7: Proceso de subida (Upload) en PlatformIO.

4. Resultados

En esta sección se documenta el funcionamiento del firmware ejecutado en el ESP32 controlando la matriz WS2812 16×16. Se incluyen fotografías y capturas de pantalla tomadas directamente de la matriz en funcionamiento. Para cada imagen se describe el modo probado y el comportamiento esperado.

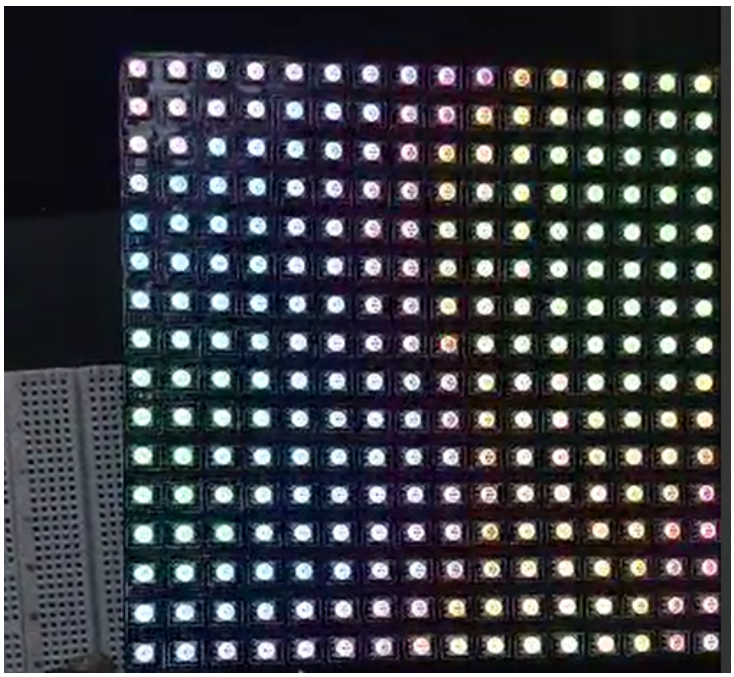


Figura 8: Modo 0 — *rainbowChase*: se espera un gradiente arcoíris que recorre la matriz; los tonos deben cambiar suavemente horizontal y verticalmente.

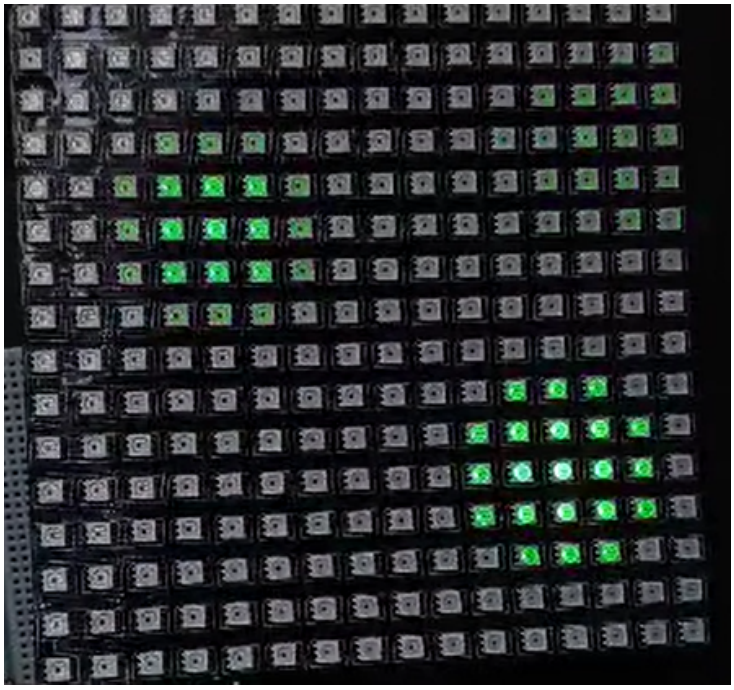


Figura 9: Modo 2 — *fireworks*: explosiones puntuales con atenuación gradual; documentar al menos una explosión y su decaimiento.

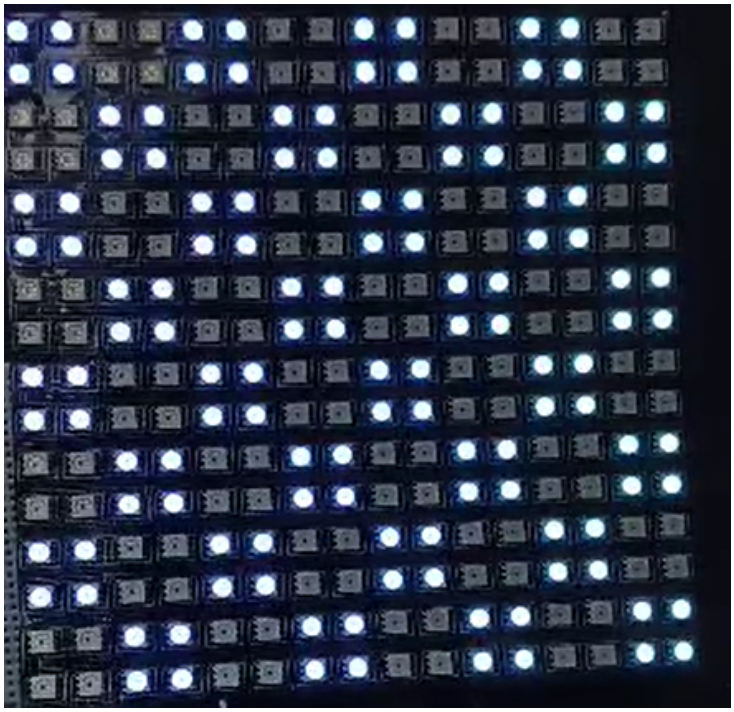


Figura 10: Modo 3 — *checkerboard*: patrón de tablero en movimiento; verificar alternancia de cuadros y desplazamiento temporal.

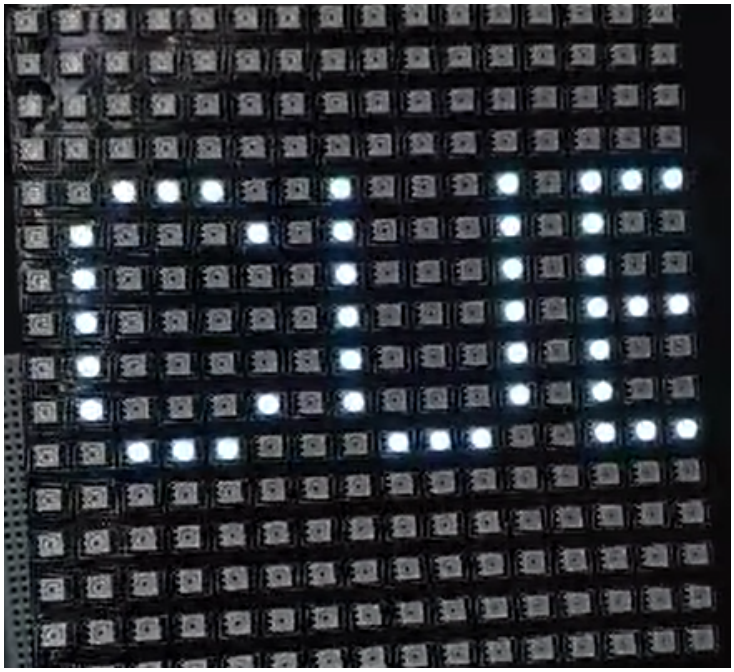


Figura 11: Modo 4 — *scrollMessage*: texto desplazándose (ej.: “UPS CUENCA”); capturar una secuencia de 3–5 frames que muestre el texto entrando, centrado y saliendo.